

Luma

Bright Living, Smart Control

Smart Home Monitoring & Control System

A Mini Project Report

Module Code: SCS 3311
Module Name: Mobile Application Design and Development
Project Type: Mini Project
Platform: Android (Kotlin) · Firebase · React Dashboard

Team Members and Responsibilities

Member 1	Android UI, Navigation, Dashboard, Reports
Member 2	Firebase, Realtime Synchronization, Device Management
Member 3	React Dashboard, Cloud Functions, Testing, Documentation

Submitted in partial fulfilment of the requirements of
Module SCS 3311 – Mobile Application Design and Development

July 11, 2026

Abstract

Luma is a smart home monitoring and control system developed as a mini project for the module SCS 3311 – Mobile Application Design and Development. The system enables a user to monitor and control household electrical devices across multiple floors and rooms of a residence through a native Android application. Because no physical Internet of Things (IoT) hardware was available for this project, a web-based Hardware Simulator, built using React, was developed to emulate the behaviour of real smart devices such as smart lights, electrical outlets, multi-switch panels, electric irons, and cameras. The Android application and the Hardware Simulator communicate indirectly through Google Firebase, which provides a Real-time Database for state synchronization, Cloud Functions for backend automation, and Cloud Messaging for push notifications. This report documents the complete design and development process of Luma, including the system architecture, technology stack, database design, user interface design, backend automation logic, testing strategy, and project management approach. The report demonstrates how a serverless, Firebase-centric architecture can be used to build a fully functional, realtime, multi-client smart home platform without the overhead of maintaining a custom backend server.

Keywords: Smart Home, Android, Firebase, Realtime Database, Cloud Functions, React, IoT Simulation, MVVM, Jetpack Compose.

Contents

Abstract	1
1 Introduction	7
1.1 Project Background	7
1.2 Motivation	7
1.3 Scope of the Project	7
1.4 Report Structure	8
2 Project Objectives	8
2.1 Primary Objectives	8
2.2 Secondary Objectives	9
3 System Architecture	9
3.1 Architectural Style	9
3.2 High-Level Architecture Diagram	9
3.3 Layered Architecture View	10
3.4 Component Responsibilities	11
3.5 Why Firebase Alone Is Sufficient	12
4 Technology Stack	12
4.1 Mobile Technologies	12
4.2 Cloud Technologies	13
4.3 Web Dashboard Technologies	14
4.4 Version Control and Deployment	14
4.5 Rejected Alternatives	15
5 Project Folder Structures	15
5.1 Android Application Folder Structure	15
5.2 React Hardware Simulator Folder Structure	16
6 Functional Requirements	17
6.1 Dashboard Requirements	17
6.2 Floor Management	18
7 Device Types and States	18
7.1 Electrical Outlet	19
7.2 Smart Light	19
7.3 Multi-Switch Panel	19
7.4 Electric Iron	19
7.5 Camera	20
7.6 Device States	20
8 Usage Reports	20
8.1 Report Types	20
8.2 Report Visualizations	21
9 Notifications	22
9.1 Notification Triggers	22
9.2 Delivery Mechanism	22
10 Hardware Simulator Dashboard	22

10.1 Purpose	23
10.2 Simulation Behaviour	23
10.3 Dashboard Layout	23
11 Firebase Database Design	24
11.1 Database Structure Overview	24
11.2 Entity-Relationship Diagram	24
11.3 JSON Schema Examples	25
11.4 Relationships Summary	26
12 Android Application Screens	26
12.1 Splash Screen	26
12.2 Dashboard	27
12.3 Floor List	27
12.4 Floor Details	27
12.5 Floor Plan	27
12.6 Device Details	28
12.7 Add Device	28
12.8 Reports	28
12.9 Notifications	28
12.10 Settings	29
13 Navigation Flow	29
13.1 Primary Navigation Path	29
13.2 Navigation Architecture	29
13.3 Use Case Diagram	29
13.4 Sequence Diagram: Controlling a Device	30
13.5 Activity Diagram: Electric Iron Safety Automation	30
13.6 UML Class Diagram	31
14 Firebase Synchronization	32
14.1 Bidirectional Synchronization Model	32
14.2 Realtime Database Listeners	33
14.3 Consistency and Conflict Handling	33
15 Firebase Cloud Functions	33
15.1 Role of Cloud Functions	33
15.2 Iron Automatic Shutdown Function	33
15.3 Other Cloud Functions	35
16 MVVM Architecture	35
16.1 Overview	35
16.2 Layer Responsibilities	36
16.3 Benefits Realized	36
17 Project Development Roadmap	37
17.1 Implementation Phases	37
17.2 Project Timeline	37
17.3 Recommended Development Order	38
18 Team Responsibilities	39
18.1 Responsibility Matrix	39

19 Testing	40
19.1 Testing Categories	40
19.2 Representative Test Cases	40
20 Security	41
20.1 Firebase Security Rules	41
20.2 Authentication	42
20.3 Realtime Security Considerations	42
20.4 Cloud Function Security	42
20.5 Database Rules Summary	42
21 Future Improvements	43
22 Conclusion	44

List of Figures

1	High-level system architecture of Luma showing the Android application, the Firebase backend, and the React Hardware Simulator as independent clients of a shared Realtime Database.	10
2	Layered view of the Luma architecture, from user-facing presentation down to backend automation.	11
3	Android application package structure.	15
4	React Hardware Simulator Dashboard folder structure.	16
5	Wireframe of the Android Dashboard screen showing the floor list.	18
6	Example bar chart of daily device running time, as shown on the Reports screen.	21
7	Example pie chart of proportional monthly power usage by device category.	21
8	Notification delivery pipeline shared by all three notification triggers.	22
9	Wireframe of the React Hardware Simulator Dashboard.	23
10	Top-level Firebase Realtime Database structure. Dashed arrow indicates a reference (device ID) rather than nested ownership.	24
11	Entity-Relationship diagram of the Luma data model.	25
12	Primary navigation flow of the Luma Android application.	29
13	Use case diagram for the Luma system, with the Homeowner as the primary actor. “Simulate Device” is exercised through the separate Hardware Simulator application during development and demonstration.	30
14	Sequence diagram for a device control action, from user tap through to the Hardware Simulator observing the resulting state change.	30
15	Activity diagram for the Electric Iron automatic-shutdown safety feature.	31
16	Simplified UML class diagram of the core Android domain and repository classes.	32
17	Synchronization path: Android application to Hardware Simulator Dashboard.	32
18	Synchronization path: Hardware Simulator Dashboard to Android application.	33
19	Flowchart of the Firebase Cloud Function implementing the Electric Iron automatic-shutdown safety feature.	34
20	MVVM architecture as applied in the Luma Android application.	36
21	Gantt-style project timeline across a 12-week mini-project schedule.	38

List of Tables

1	Responsibilities of each architectural component.	11
2	Mobile technology choices and their trade-offs.	13
3	Cloud technology choices and their trade-offs.	14
4	Web dashboard technology choices and their trade-offs.	14
5	Version control and deployment options considered.	15
6	Explanation of Android application folders.	16
7	Explanation of React Hardware Simulator folders.	17
8	Device states and the conditions under which each occurs.	20
9	Responsibilities of each MVVM layer in the Luma Android application.	36
10	Thirteen-phase implementation roadmap for Luma.	37
11	Recommended development order with milestones and expected outputs.	39
12	High-level responsibility allocation across the three team members.	39
13	Responsibility matrix: P = Primary owner, S = Secondary support.	40
14	Testing categories applied to the Luma project.	40
15	Representative test cases and expected results.	41
16	Summary of database access rules by node.	43
17	Identified future improvements beyond the current project scope.	43

1 Introduction

1.1 Project Background

Modern residential buildings increasingly rely on connected electrical devices to improve comfort, safety, and energy efficiency. A typical smart home solution consists of physical IoT devices, a communication protocol, a cloud backend, and one or more client applications that allow the homeowner to observe and control the state of the house. Building and provisioning genuine IoT hardware, however, is expensive and outside the practical scope of a university mini project delivered within a single semester.

Luma addresses this constraint by separating the *control and monitoring experience* from the *physical device layer*. Instead of real hardware, Luma uses a purpose-built **Hardware Simulator**, a standalone React web application that behaves exactly like a collection of real IoT devices from the point of view of the rest of the system. The Android application and the Hardware Simulator never communicate directly with one another. Instead, both are independent clients of a shared **Firebase Realtime Database**, which acts as the single source of truth for the state of every floor, room, and device in the simulated home. Firebase Cloud Functions observe changes to this database and perform backend automation such as automatically switching off an electric iron after a maximum operating duration, while Firebase Cloud Messaging delivers push notifications to the Android application when safety-relevant events occur.

This architecture allows the project to demonstrate every core competency expected of a mobile application design and development module – native Android development, MVVM architecture, realtime data synchronization, cloud backend integration, and multi-client system design – without requiring physical hardware, a dedicated server, or a custom REST API.

1.2 Motivation

The motivation behind Luma is threefold:

- (i) **Educational value.** The project is designed to exercise the full breadth of the mobile application development syllabus: UI/UX design, state management, cloud integration, background automation, and testing.
- (ii) **Practical realism.** Rather than building a simplified toy application, Luma mirrors the architecture of real commercial smart home platforms (such as Google Home or Samsung SmartThings), which also rely on a cloud intermediary between the control application and the physical devices.
- (iii) **Feasibility within project constraints.** By replacing physical hardware with a software simulator and replacing a custom backend with Firebase’s managed services, the team could focus engineering effort on application logic and user experience rather than on infrastructure and hardware procurement.

1.3 Scope of the Project

The scope of Luma covers:

- A native Android application (Kotlin, Jetpack Compose, MVVM) for monitoring and controlling devices.
- A Firebase Realtime Database schema representing users, floors, rooms, devices, alerts, and reports.
- Firebase Cloud Functions for safety automation (e.g. automatic iron shutdown) and report generation triggers.

- Firebase Cloud Messaging for push notifications.
- A React-based Hardware Simulator Dashboard that emulates physical smart devices for demonstration and testing purposes.
- Usage reporting (daily, weekly, monthly) derived from device state history.

The project does *not* include real hardware integration, voice assistant integration, or a custom backend server; these are documented as future improvements in Section 21.

1.4 Report Structure

The remainder of this report is organized as follows. Section 2 states the project objectives. Section 3 presents the system architecture. Section 4 justifies the technology stack. Section 5 documents the project folder structures. Section 6 details functional requirements. Section 7 describes device types and states. Section 8 covers usage reporting. Section 9 covers notifications. Section 10 describes the Hardware Simulator. Section 11 presents the Firebase database design. Section 12 documents the Android application screens. Section 13 explains navigation flow. Section 14 explains realtime synchronization. Section 15 explains Cloud Functions automation. Section 16 explains the MVVM architecture. Section 17 presents the development roadmap. Section 18 documents team responsibilities. Section 19 presents the testing strategy. Section 20 discusses security. Section 21 discusses future improvements, and Section 22 concludes the report.

2 Project Objectives

The objectives of Luma were defined to cover the complete lifecycle of a smart home platform, from monitoring through to control, automation, and reporting.

2.1 Primary Objectives

01. **Smart Home Monitoring.** Provide the user with a live, always-up-to-date view of the state of every device in the home, including its power state, connectivity status, and any active alerts.
02. **Smart Home Control.** Allow the user to change the state of any controllable device (turn on/off, schedule, configure) directly from the Android application, with the change reflected across all connected clients within a sub-second latency in typical conditions.
03. **Multiple Floor Management.** Support an arbitrary number of floors per home, each containing its own set of rooms and devices, so that the system scales realistically to multi-storey residences.
04. **Device Management.** Allow devices to be added, edited, removed, and reassigned between rooms and floors, with each device type exposing an appropriate set of controls.
05. **Real-Time Synchronization.** Guarantee that any state change made on one client (Android app or Hardware Simulator) is immediately visible on all other connected clients, using Firebase Realtime Database listeners.
06. **Cloud-Based Architecture.** Avoid a single point of failure or a custom-maintained server by relying entirely on Google’s managed Firebase infrastructure for data storage, automation, and messaging.

- O7. **Safety Automation.** Automatically enforce safety constraints, most notably the automatic shutdown of an electric iron after a configurable maximum operating duration, without requiring user intervention.
- O8. **Usage Reporting.** Aggregate device state history into daily, weekly, and monthly usage reports that help the user understand power consumption patterns and device activity.
- O9. **Hardware Simulation.** Provide a realistic, interactive substitute for physical IoT hardware so that the full system can be demonstrated and tested end-to-end without physical devices.
- O10. **User-Friendly Interface.** Present all functionality through a clean, modern, accessible interface built with Jetpack Compose on Android and Material UI on the web dashboard.

2.2 Secondary Objectives

- Demonstrate correct application of the MVVM architectural pattern in a Kotlin/Jetpack Compose codebase.
- Demonstrate the use of Firebase Cloud Functions as a lightweight, event-driven backend, eliminating the need for a custom REST API.
- Provide a codebase that is modular, testable, and maintainable, with a clear separation between UI, business logic, and data layers.
- Provide comprehensive documentation suitable for academic evaluation and for future extension of the project.

3 System Architecture

3.1 Architectural Style

Luma follows a **serverless, Backend-as-a-Service (BaaS) architecture**. Rather than hosting a custom REST API on a server that the team would need to provision, secure, and maintain, all backend responsibilities are delegated to Google Firebase. The Android application and the React Hardware Simulator are both thin clients that read and write directly to the Firebase Realtime Database through the official Firebase SDKs. Firebase Cloud Functions provide the only piece of custom backend logic required, executing in response to database triggers rather than HTTP requests from the clients.

3.2 High-Level Architecture Diagram

Figure 1 shows the overall data flow between the three major components of the system.

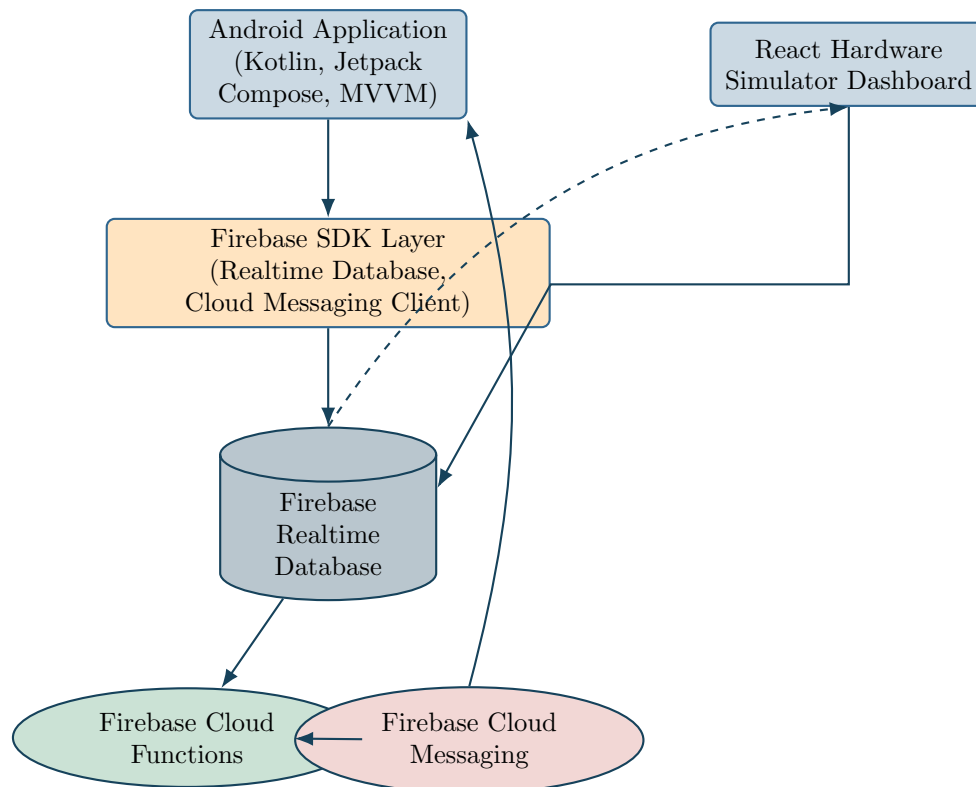


Figure 1: High-level system architecture of Luma showing the Android application, the Firebase backend, and the React Hardware Simulator as independent clients of a shared Realtime Database.

The dashed arrow indicates that state changes written by the Hardware Simulator into the Realtime Database are pushed back out to the Android application (and vice versa) through Firebase’s realtime listener mechanism, without either client ever contacting the other directly.

3.3 Layered Architecture View

Figure 2 presents the same system as a layered stack, emphasizing the separation between the presentation layer, the synchronization layer, the cloud data layer, and the automation layer.

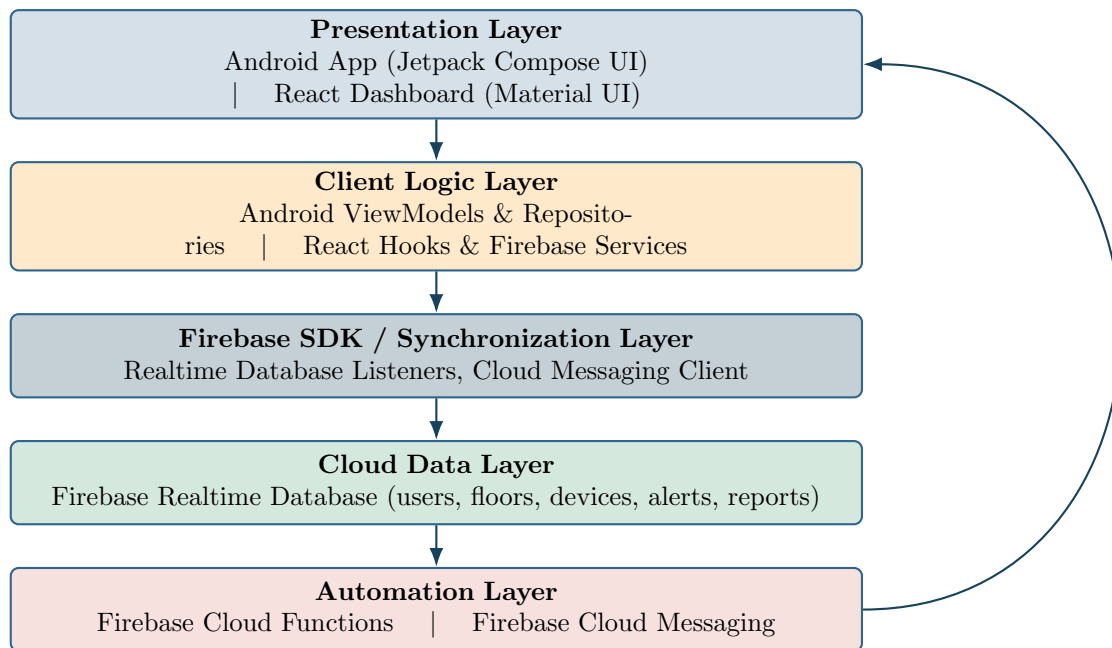


Figure 2: Layered view of the Luma architecture, from user-facing presentation down to backend automation.

3.4 Component Responsibilities

Component		Responsibility
Android	Applica- tion	Presents floors, rooms, and devices to the homeowner; allows control actions; displays reports and notifications; observes the Realtime Database for live updates.
Firebase	SDK	Provides the client libraries used by both the Android app and the React dashboard to read, write, and subscribe to changes in the Realtime Database, and to register for Cloud Messaging.
Firebase	Realtime Database	Stores the canonical state of the entire home (floors, rooms, devices, alerts, reports) as a JSON tree; pushes incremental updates to all subscribed clients in real time.
Firebase	Cloud Functions	Executes server-side logic in response to database writes – for example, monitoring how long an electric iron has been switched on and automatically turning it off after the configured maximum duration.
Firebase	Cloud Messaging	Delivers push notifications to the Android application when Cloud Functions detect safety-relevant events (e.g. automatic shutdown, device disconnection, device error).
React	Hardware Simulator	A standalone web application that emulates the physical behaviour of smart devices (lights, outlets, switch panels, irons, cameras), writing simulated sensor and state data into the Real-time Database exactly as real hardware would.

Table 1: Responsibilities of each architectural component.

3.5 Why Firebase Alone Is Sufficient

A common design question is whether Luma requires a custom backend (for example, built with FastAPI, Node.js/Express, Spring Boot, or Django) in addition to Firebase. The team deliberately decided against this for the following reasons:

- (a) **Realtime Database already provides the required data layer.** All entities in Luma (floors, rooms, devices, alerts, reports) are naturally represented as a JSON tree, which is exactly the data model that the Firebase Realtime Database is optimized for. A relational or document database behind a custom API would add complexity without adding capability.
- (b) **Cloud Functions replace the need for a REST API.** The only server-side logic Luma requires – automatic iron shutdown, alert generation, and report aggregation – is naturally expressed as a reaction to a database write. Firebase Cloud Functions support exactly this trigger-based execution model, removing the need to design, host, and secure a separate REST API.
- (c) **Built-in realtime synchronization.** A custom backend would require the team to implement WebSockets or a polling mechanism to achieve the same live-update behaviour that the Firebase Realtime Database provides out of the box.
- (d) **Reduced operational overhead.** Firebase is a fully managed service; there is no server to provision, patch, or scale. This is particularly appropriate for a mini project with a fixed, short development timeline and no dedicated DevOps resource.
- (e) **Built-in security rules.** Firebase Security Rules provide declarative, per-node access control directly on the database, removing the need to implement authentication and authorization middleware in a custom API layer.
- (f) **Cost and simplicity.** Firebase’s free tier is more than sufficient for a mini project of this scale, whereas hosting a custom backend would introduce additional cost and configuration overhead with no corresponding benefit to the project’s learning objectives.

The trade-off of this decision is discussed further in Section 4: a Firebase-only architecture is less portable to a non-Google cloud provider and offers less flexibility for complex relational queries than a traditional backend would, but for the scope of Luma these limitations are outweighed by the benefits above.

4 Technology Stack

4.1 Mobile Technologies

Technology	Advantages	Disadvantages / Trade-offs
Kotlin	Official first-class language for Android; concise, null-safe syntax; full interoperability with existing Java libraries; strong coroutine support for asynchronous Firebase calls.	Slightly steeper learning curve than Java for developers unfamiliar with functional-style syntax.

Technology	Advantages	Disadvantages / Trade-offs
Android Studio	Official Android IDE with integrated emulator, layout inspector, Firebase Assistant plug-in, and Gradle build management.	Resource-intensive; requires a reasonably powerful development machine.
Jetpack Compose	Declarative UI toolkit that reduces boilerplate compared to XML layouts; integrates cleanly with ViewModel state via State and Flow ; enables rapid iteration on the Dashboard, Floor Plan, and Device screens.	Relatively newer toolkit; some third-party libraries still offer better support for the legacy View system.
MVVM Architecture	Clear separation of UI, state-holding, and data-access logic; recommended by Google for Compose applications; simplifies unit testing of business logic in isolation from the UI.	Requires additional boilerplate (ViewModel, UiState classes) for very simple screens.

Table 2: Mobile technology choices and their trade-offs.

4.2 Cloud Technologies

Technology	Advantages	Disadvantages / Trade-offs
Firebase Realtime Database	Low-latency, JSON-tree real-time synchronization out of the box; simple SDK for both Android and JavaScript; ideal for frequently changing device state.	Limited support for complex relational queries compared to Cloud Firestore or SQL.
Firebase Cloud Functions	Event-driven, serverless execution triggered directly by database writes; removes the need for a custom REST API or server hosting.	Cold-start latency on the free tier; execution time limits on long-running tasks.
Firebase Cloud Messaging	Reliable, Google-managed push notification delivery to Android devices, including when the app is in the background.	Requires correct handling of notification permissions on Android 13+.
Firebase Authentication (Optional)	Provides ready-made email/password and Google sign-in flows, tightly integrated with Realtime Database Security Rules.	Adds a dependency the mini project can optionally omit if a single shared household account is acceptable for demonstration purposes.

Technology	Advantages	Disadvantages / Trade-offs
------------	------------	----------------------------

Table 3: Cloud technology choices and their trade-offs.

4.3 Web Dashboard Technologies

Technology	Advantages	Disadvantages / Trade-offs
React	Component-based architecture maps naturally onto device cards, floor lists, and status chips; large ecosystem of hooks for Firebase integration.	Requires careful state management as the number of simulated devices grows.
Vite	Extremely fast development server and hot module replacement, well suited for rapid iteration on the simulator UI during testing.	Smaller plug-in ecosystem than older bundlers such as Webpack, though sufficient for this project's needs.
Material UI	Ready-made, accessible component library (cards, chips, switches, app bars) that accelerates dashboard construction and produces a professional look with minimal custom CSS.	Visual style can appear generic unless themed; theming was applied to match the Luma brand palette.

Table 4: Web dashboard technology choices and their trade-offs.

4.4 Version Control and Deployment

Technology	Advantages	Disadvantages / Trade-offs
Git	Industry-standard distributed version control; enables the three-member team to work in parallel on Android, Firebase, and React codebases.	Requires discipline around branching and merge conflict resolution.
GitHub	Central remote repository, issue tracking, and pull-request based code review for the team.	None significant for a project of this scale.
Firebase Hosting	Free, HTTPS-enabled static hosting that integrates naturally with the rest of the Firebase project, ideal for deploying the React Hardware Simulator Dashboard.	Limited to static/serverless hosting; not required for this project since Cloud Functions cover backend logic.

Technology	Advantages	Disadvantages / Trade-offs
Vercel (Alternative)	Zero-configuration deployment for React/Vite projects with automatic preview deployments per pull request.	Introduces a second cloud provider alongside Firebase; the team ultimately preferred Firebase Hosting for a single-provider deployment story.

Table 5: Version control and deployment options considered.

4.5 Rejected Alternatives

The team explicitly considered and rejected the following approaches:

- **FastAPI / Node.js (Express) / Spring Boot / Django backend.** A custom REST API would duplicate functionality already provided by Firebase Realtime Database and Cloud Functions, while adding server hosting, authentication middleware, and API versioning overhead with no corresponding benefit for a project of this scope.
- **Cloud Firestore instead of Realtime Database.** Firestore offers richer querying but has higher per-write latency characteristics for very frequent small updates (such as a device toggling state many times during a demonstration); the simple JSON-tree model of the Realtime Database better matches Luma’s data shape.
- **Native Android Views (XML layouts) instead of Jetpack Compose.** Compose was selected for its declarative model, which reduces boilerplate and pairs naturally with ViewModel-driven state in an MVVM architecture.

5 Project Folder Structures

5.1 Android Application Folder Structure

Figure 3 shows the package structure of the Android application, and Table 6 explains the purpose of each package.

```

app/
  ui/
    dashboard/
    floor/
    device/
    report/
    settings/
  model/
  repository/
  firebase/
  viewmodel/
  navigation/
  utils/
  MainActivity.kt

```

Figure 3: Android application package structure.

Package	Purpose
ui/	Root package for all Jetpack Compose UI code, organized by feature.
ui/dashboard/	Composables for the main Dashboard screen listing all floors.
ui/floor/	Composables for Floor List, Floor Details, and the interactive Floor Plan grid.
ui/device/	Composables for Device Details and Add Device screens, including per-device-type control widgets.
ui/report/	Composables for the Reports screen, including usage charts.
ui/settings/	Composables for the Settings screen.
model/	Kotlin data classes representing domain entities: Floor , Room , Device , Alert , Report , and device-specific subtypes.
repository/	Repository classes that abstract Firebase access behind a clean Kotlin API consumed by ViewModels, following the Repository pattern.
firebase/	Thin wrapper classes around the Firebase Realtime Database and Cloud Messaging SDKs, including listener registration and serialization helpers.
viewmodel/	ViewModel classes (one per screen/feature) that expose UI state as Kotlin StateFlow and handle user actions.
navigation/	Navigation graph definition using Jetpack Navigation Compose, including route definitions and argument passing.
utils/	Shared utility functions (date/time formatting, unit conversion, validation helpers).
MainActivity.kt	Application entry point; hosts the Compose NavHost and initializes Firebase.

Table 6: Explanation of Android application folders.

5.2 React Hardware Simulator Folder Structure

```
src/
  components/
  pages/
  firebase/
  hooks/
  utils/
  App.jsx
  main.jsx
```

Figure 4: React Hardware Simulator Dashboard folder structure.

Folder / File	Purpose
components/	Reusable presentational components: Sidebar, Navbar, Device Card, Status Chip, Toggle Button, Floor Selector.
pages/	Top-level route components: Floor Overview page, Device Detail page, Camera Feed page.
firebase/	Firebase configuration and thin service functions for reading/writing device state to the Realtime Database.

Folder / File	Purpose
hooks/	Custom React hooks (e.g. <code>useDeviceState</code> , <code>useFloorList</code>) that subscribe to Realtime Database listeners and expose live data to components.
utils/	Formatting and simulation-logic helpers, such as generating mock camera snapshots or simulated sensor jitter.
App.jsx	Root application component defining the overall dashboard layout and routing.
main.jsx	Vite/React entry point that mounts <code>App.jsx</code> into the DOM.

Table 7: Explanation of React Hardware Simulator folders.

6 Functional Requirements

6.1 Dashboard Requirements

The Dashboard is the first substantive screen the user sees after the Splash Screen. It lists every floor currently defined for the home and allows floor-level management.

- **View all floors.** The Dashboard displays a scrollable list/grid of floor cards, each showing the floor name, a thumbnail icon, the number of rooms, and a summary count of devices that are currently ON.
- **Add floor.** A floating action button opens a dialog in which the user enters a floor name (e.g. “Ground Floor”, “First Floor”) and selects an icon; on confirmation, a new floor node is created in the Realtime Database.
- **Delete floor.** A long-press or overflow menu on a floor card opens a confirmation dialog; on confirmation, the floor node and all of its child rooms/devices are removed from the database.
- **Edit floor.** The user can rename a floor or change its icon through the same dialog used for adding a floor, pre-populated with the existing values.
- **Select floor.** Tapping a floor card navigates to the Floor Details screen for that floor.

Figure 5 shows a wireframe of the Dashboard screen.

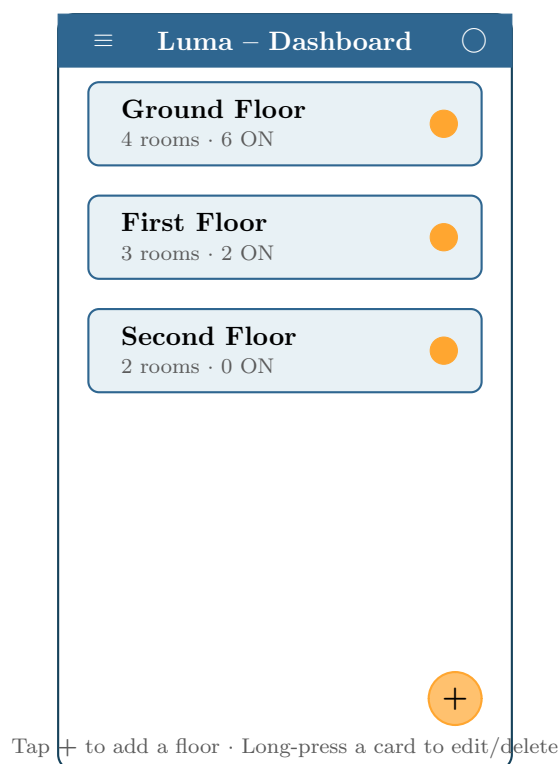


Figure 5: Wireframe of the Android Dashboard screen showing the floor list.

6.2 Floor Management

Each floor contains one or more rooms, and each room contains one or more devices. The Floor Details screen presents rooms as an interactive grid, and selecting a room opens an interactive floor plan.

- **Rooms.** Rooms are simple named containers (e.g. “Living Room”, “Kitchen”, “Master Bedroom”) belonging to a floor.
- **Devices.** Each room contains zero or more devices, each with a type (outlet, light, switch panel, iron, camera), a name, and a current state.
- **Grid layout.** Rooms are displayed using a responsive grid (`LazyVerticalGrid` in Jetpack Compose) so that the number of visible room cards adapts to screen size.
- **Interactive floor plan.** Selecting a room opens a simplified 2D floor-plan view where devices are represented as tappable icons positioned approximately where they are physically located in the room, giving the user spatial context in addition to the flat device list.

Implementation notes. The Floor Details ViewModel subscribes to the `/floors/{floorId}/rooms` node of the Realtime Database. Each room composable further subscribes to `/floors/{floorId}/rooms/{roomId}` so that device state changes are reflected immediately without requiring the user to refresh the screen. The floor-plan coordinates for each device are stored as normalized `x`, `y` fields (0.0–1.0) on the device record so that the layout scales correctly to any screen size.

7 Device Types and States

Luma supports five categories of simulated smart devices. Each device type is modelled as a Kotlin sealed class on the Android side and as a corresponding JavaScript object shape on

the React Hardware Simulator side, sharing a common JSON representation in the Realtime Database.

7.1 Electrical Outlet

- **ON / OFF.** A single toggle switch controls whether the outlet is supplying power.
- **Realtime synchronization.** Toggling the outlet from either the Android app or the Hardware Simulator updates the `state` field of the device node, and the change propagates to all connected clients within the Realtime Database's typical sub-second latency.

The Electrical Outlet is the simplest device type in Luma and serves as the baseline example for the ON/OFF control pattern reused, in extended form, by the other device types.

7.2 Smart Light

- **ON / OFF.** Manual toggle control, identical in behaviour to the Electrical Outlet.
- **Automatic scheduling.** The user can define a `startTime` and `endTime` (stored as HH:mm strings) after which a Cloud Function automatically turns the light on and off, simulating a timer-based automation rule.
- **Usage tracking.** Every ON→OFF transition is appended to the device's `history` node with a timestamp, enabling the Reports feature (Section 8) to compute total running time for the light.

7.3 Multi-Switch Panel

- **Configurable switch count.** A single Multi-Switch Panel device can be configured with 2, 3, or 5 individual switches, matching common physical wall-panel form factors.
- **Independent control.** Each switch within the panel has its own `state` field (ON/OFF) and can be toggled independently of the others; the panel as a whole has no single combined state.
- **Data representation.** Switches are stored as a keyed sub-map (`switch1`, `switch2`, ...) under the panel's device node, as shown in Section 11.

7.4 Electric Iron

- **ON / OFF.** Manual toggle control initiates or ends an ironing session.
- **Maximum operating duration.** A configurable maximum duration (default 30 minutes) defines the longest period the iron may remain switched on continuously, reflecting the fire-safety rationale for this feature.
- **Automatic shutdown.** A Firebase Cloud Function (detailed in Section 15) monitors the elapsed time since the iron was turned on and force-sets its state to OFF once the maximum duration is exceeded.
- **Push notification.** When the automatic shutdown occurs, a Cloud Function triggers a Firebase Cloud Messaging notification informing the user that the iron was switched off automatically for safety.
- **Realtime synchronization.** As with all devices, the resulting OFF state is immediately visible on both the Android app and the Hardware Simulator.

The Electric Iron is the primary demonstration of Luma’s *Safety Automation* objective (Section 2).

7.5 Camera

- **Snapshot.** The user can request a still image representing the camera’s current view.
- **Mock image.** Because no physical camera exists, the Hardware Simulator returns a pre-defined mock image (or a procedurally generated placeholder) rather than a live video feed.
- **Refresh image.** The Android app can request a new snapshot at any time, which updates the `lastSnapshotUrl` and `lastSnapshotAt` fields on the device node.
- **Connectivity states.** A camera device additionally exposes `online`, `offline`, and `disconnected` states, distinct from the simple ON/OFF model used by the other device types, since a camera’s usefulness depends primarily on its connectivity rather than a power toggle.

7.6 Device States

Every device in Luma, regardless of type, exposes a `status` field in addition to its type-specific `state` fields. Table 8 defines the possible values of this field and when each occurs.

State	When It Occurs
ON	The device (or, for a Multi-Switch Panel, an individual switch) is actively powered/active, either because the user switched it on manually, a schedule activated it, or a Cloud Function turned it on.
OFF	The device is intentionally powered down, either through manual user action, the end of a schedule, or an automatic safety shutdown (e.g. the Electric Iron’s maximum-duration rule).
ERROR	The Hardware Simulator has flagged the device as reporting an invalid or unexpected internal condition (for example, simulated sensor failure); the Android app surfaces this with a red status chip and generates an alert.
DISCONNECTED	The device has not written a heartbeat/state update within the expected interval, simulating a device that has lost network connectivity; this state is periodically evaluated by a Cloud Function that compares the device’s <code>lastSeen</code> timestamp against the current time.

Table 8: Device states and the conditions under which each occurs.

8 Usage Reports

The Reports feature aggregates the `history` sub-nodes maintained for every device into summaries that help the user understand how the home is being used.

8.1 Report Types

- **Daily Usage.** Total ON duration per device for the current calendar day, refreshed as new history entries arrive.

- **Weekly Usage.** Total ON duration per device, aggregated over the last seven days, useful for spotting week-over-week trends.
- **Monthly Usage.** Total ON duration per device, aggregated over the current calendar month.
- **Running Time.** The cumulative number of hours a given device has been active, since installation or since a user-selected reset point.
- **Power Usage.** An estimated energy consumption figure (kWh) computed by multiplying running time by a configurable nominal wattage per device type.
- **Safety Events.** A log of automatic-shutdown and disconnection events (e.g. iron auto-off events, device disconnections), each with a timestamp and the device involved.

8.2 Report Visualizations

Bar charts are used to compare running time across devices within a room or floor, while pie charts are used to show the proportional contribution of each device to total power usage. Figures 6 and 7 show representative examples of the intended chart styles.

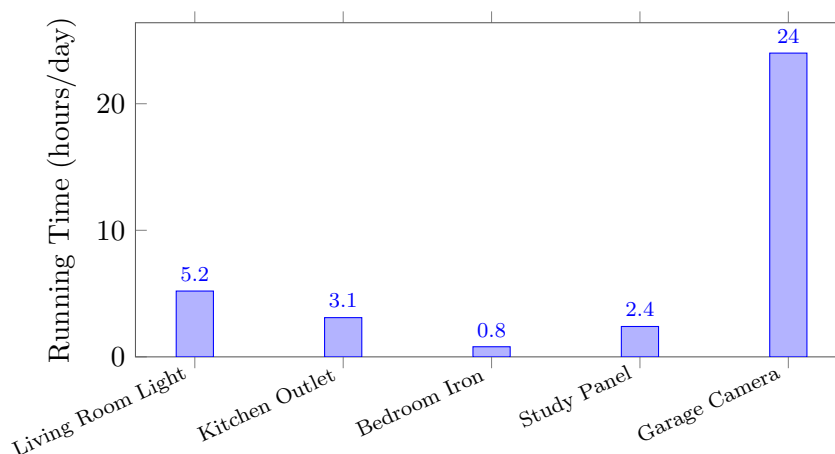


Figure 6: Example bar chart of daily device running time, as shown on the Reports screen.

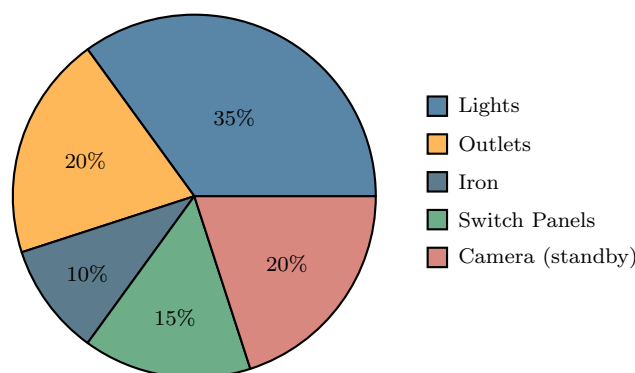


Figure 7: Example pie chart of proportional monthly power usage by device category.

Implementation note. The pie chart in Figure 7 is illustrative of the intended chart type; the production Android implementation renders equivalent bar and pie charts natively using a Compose charting library, with data sourced from the `ReportViewModel`, which aggregates the `/reports` node described in Section 11.

9 Notifications

Notifications keep the user informed of safety-relevant and connectivity-relevant events without requiring them to keep the app open.

9.1 Notification Triggers

- **Iron automatically turned OFF.** Raised when the maximum-operating-duration Cloud Function forces an Electric Iron to OFF (Section 15).
- **Device disconnected.** Raised when a scheduled Cloud Function detects that a device's `lastSeen` timestamp has exceeded the disconnection threshold (default 60 seconds in the simulated environment).
- **Device error.** Raised when the Hardware Simulator writes an `ERROR` status for a device, simulating an internal fault.

9.2 Delivery Mechanism

- **Cloud notification.** All notification triggers first write a structured entry into the `/alerts` node of the Realtime Database, giving the Android app an in-app notification history even if the device was offline when the event occurred.
- **Push notification.** In parallel, the Cloud Function that detected the event calls the Firebase Cloud Messaging Admin SDK to deliver a push notification to the registered device token(s) for the household, ensuring the user is alerted even when the Android app is in the background or closed.

Figure 8 illustrates the notification pipeline for the iron auto-shutdown scenario, which is representative of all three trigger types.

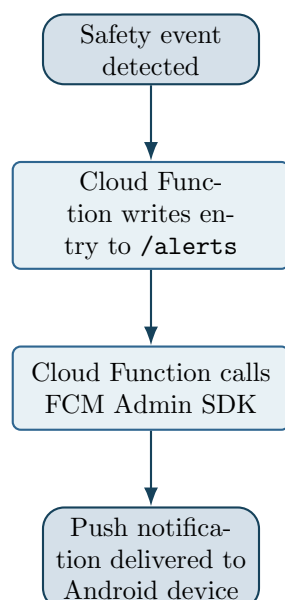


Figure 8: Notification delivery pipeline shared by all three notification triggers.

10 Hardware Simulator Dashboard

10.1 Purpose

The Hardware Simulator Dashboard is a completely separate React web application; it is **not** part of the Android application and is not distributed to the end user. Its sole purpose is to stand in for physical IoT hardware during development and demonstration, by writing realistic, time-varying state data into the same Firebase Realtime Database that the Android application reads from. From the perspective of the Realtime Database and the Cloud Functions, data written by the simulator is indistinguishable from data that would be written by real hardware, which allows the rest of the system to be developed and evaluated as if physical devices were present.

10.2 Simulation Behaviour

- Each simulated device periodically writes a **lastSeen** heartbeat timestamp, allowing the disconnection-detection Cloud Function to be exercised realistically.
- Toggling a device card in the simulator writes directly to the corresponding device's **state** field, exactly as the Android app does, demonstrating bidirectional synchronization.
- The simulator can be manually instructed (via a developer control) to simulate a device entering the **ERROR** or **DISCONNECTED** state, to test the Android app's handling of these states without waiting for a real fault.
- The Camera device type returns a mock image on snapshot request rather than a live feed, consistent with the absence of real camera hardware.

10.3 Dashboard Layout

Figure 9 shows a wireframe of the Hardware Simulator Dashboard layout.

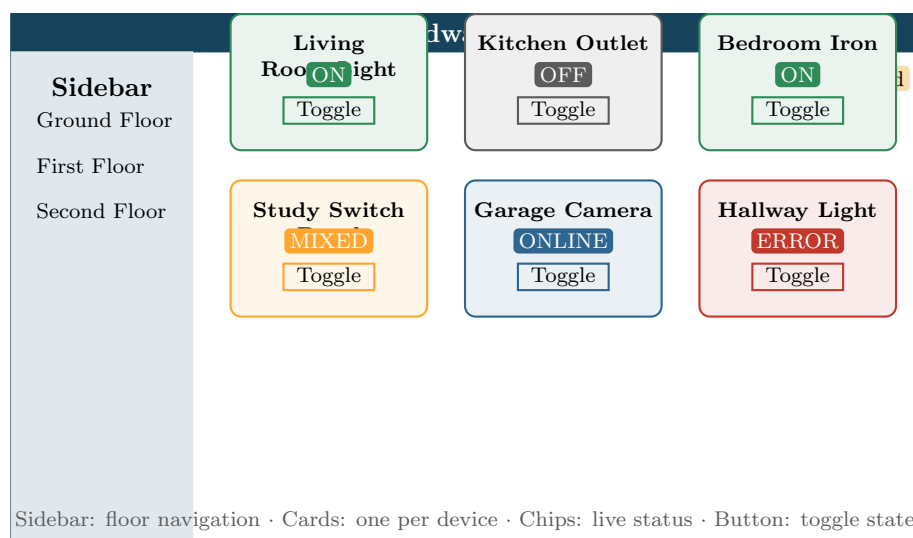


Figure 9: Wireframe of the React Hardware Simulator Dashboard.

The layout consists of a **Navbar** across the top showing the simulator title and Firebase connection status, a **Sidebar** listing floors for navigation, a main content area of **Device Cards** (one per device) each showing a **Status Chip** reflecting the current device state and a **Toggle Button** for manual control, and a live realtime-update indicator confirming that the dashboard is subscribed to the Realtime Database.

11 Firebase Database Design

11.1 Database Structure Overview

The Realtime Database is organized as a single JSON tree with five top-level nodes: **users**, **floors**, **devices**, **alerts**, and **reports**. Devices are stored in a flat top-level node (rather than nested inside floors/rooms) and referenced by ID from within the floor/room structure, which is a deliberate denormalization that keeps individual device listeners lightweight, following Firebase’s recommended best practice of flattening deeply nested data.

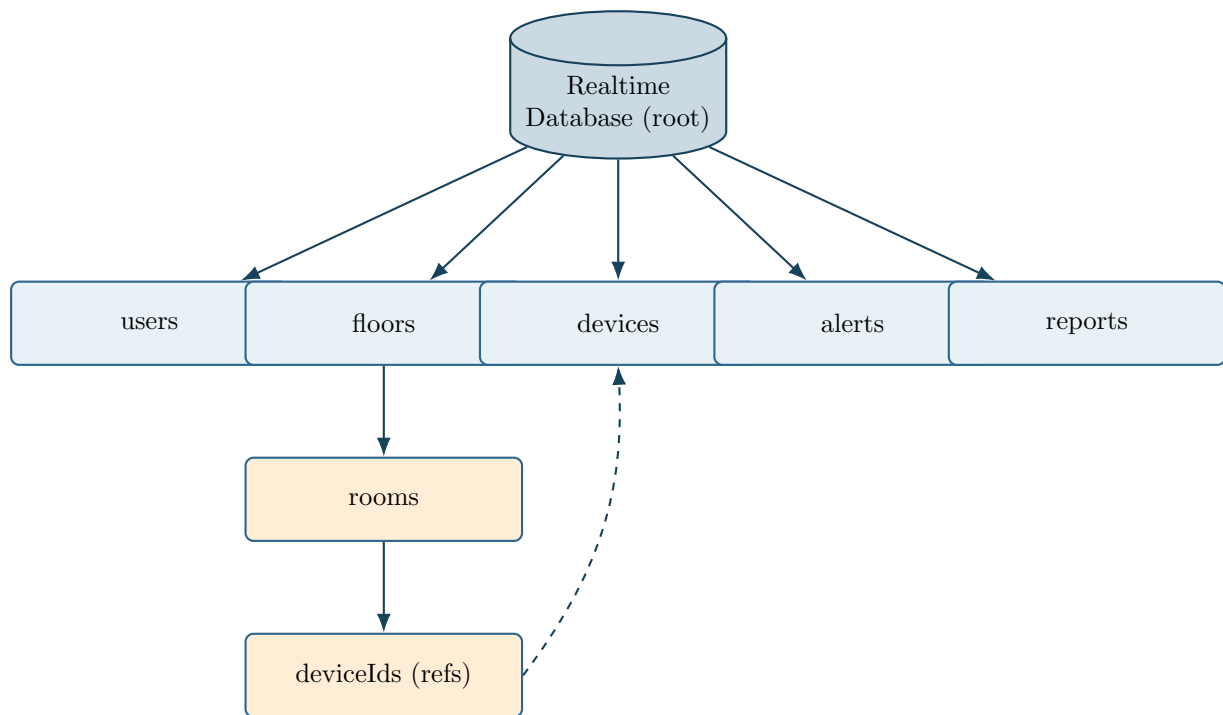


Figure 10: Top-level Firebase Realtime Database structure. Dashed arrow indicates a reference (device ID) rather than nested ownership.

11.2 Entity-Relationship Diagram

Although the Realtime Database is non-relational, the logical relationships between entities are best understood through the ER-style diagram in Figure 11.

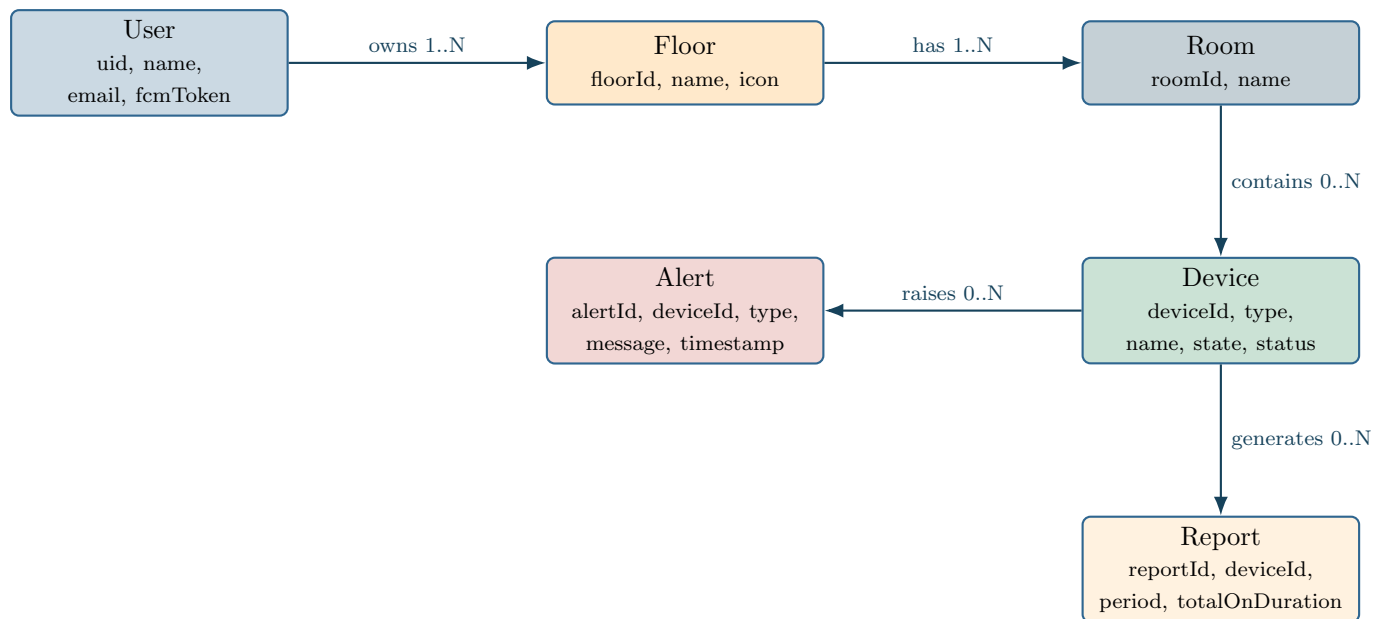


Figure 11: Entity-Relationship diagram of the Luma data model.

11.3 JSON Schema Examples

```

1 "floors": {
2   "floor001": {
3     "name": "Ground Floor",
4     "icon": "home",
5     "rooms": {
6       "room001": {
7         "name": "Living Room",
8         "devices": {
9           "device001": true,
10          "device002": true
11        }
12      }
13    }
14  }
15 }
  
```

Listing 1: Example JSON structure for a single floor with one room and a device reference.

```

1 "devices": {
2   "device001": {
3     "type": "iron",
4     "name": "Bedroom Iron",
5     "state": "ON",
6     "status": "ON",
7     "turnedOnAt": 1739000000000,
8     "maxDurationMinutes": 30,
9     "lastSeen": 1739000590000
10  },
11  "device002": {
12    "type": "switchPanel",
13    "name": "Study Switch Panel",
14    "switchCount": 3,
15    "switches": {
16      "switch1": "ON",
17      "switch2": "OFF",
18      "switch3": "OFF"
19    }
20  }
21 }
  
```

```

19   },
20   "status": "ON",
21   "lastSeen": 1739000600000
22 }
23 }

```

Listing 2: Example JSON structure for devices, including an Electric Iron and a Multi-Switch Panel.

```

1  "alerts": {
2    "alert001": {
3      "deviceId": "device001",
4      "type": "AUTO_SHUTDOWN",
5      "message": "Iron automatically turned OFF after 30 minutes",
6      "timestamp": 1739001800000
7    }
8  },
9  "reports": {
10   "device001_2026-07-10": {
11     "deviceId": "device001",
12     "period": "daily",
13     "date": "2026-07-10",
14     "totalOnDurationMinutes": 48,
15     "estimatedKwh": 0.6
16   }
17 }

```

Listing 3: Example JSON structure for an alert and a report entry.

11.4 Relationships Summary

- A **User** owns one or more **Floors** (a single-household deployment in this mini project owns all floors; multi-user support is discussed as a future improvement in Section 21).
- A **Floor** contains one or more **Rooms**.
- A **Room** contains zero or more **Devices**, referenced by ID rather than nested directly, keeping the `devices` node flat and each device's listener lightweight.
- A **Device** may raise zero or more **Alerts** over its lifetime.
- A **Device** generates one **Report** entry per aggregation period (daily/weekly/monthly).

12 Android Application Screens

This section documents the ten primary screens of the Luma Android application. For each screen, the purpose, key components, navigation behaviour, expected behaviour, user interactions, and possible future improvements are described. Wireframes are provided for the most structurally significant screens; the Dashboard and Hardware Simulator wireframes were already presented in Figures 5 and 9.

12.1 Splash Screen

Purpose. Displays the Luma logo and tagline while Firebase initializes and an initial authentication/connectivity check is performed.

Components. Centered logo, tagline text, indeterminate progress indicator.

Navigation. Automatically navigates to the Dashboard after initialization completes (typically under one second); no back-navigation is possible from this screen.

Expected behaviour. If Firebase initialization fails (e.g. no network connectivity), an inline retry message is shown instead of navigating forward.

User interactions. None beyond an optional retry tap on failure.

Future improvements. Add an animated logo reveal and a persisted-session check to skip directly to a previously selected floor.

12.2 Dashboard

Purpose. Entry point for floor-level monitoring and management, as detailed in Section 6.

Components. Top app bar, scrollable floor card list, floating action button for adding a floor.

Navigation. Tapping a floor card navigates to Floor Details; the overflow menu in the app bar links to Settings and Notifications.

Expected behaviour. The floor list updates live as devices under any floor change state (e.g. the “ON count” badge on each card).

User interactions. Tap to select a floor, long-press or overflow-menu tap to edit/delete a floor, FAB tap to add a floor.

Wireframe. See Figure 5.

Future improvements. Add drag-to-reorder floors and a home-wide “all devices off” quick action.

12.3 Floor List

Purpose. An alternate, denser list view of all floors, accessible from the navigation drawer, intended for homes with many floors.

Components. Compact list rows (icon, name, room count), search/filter field.

Navigation. Tapping a row navigates to Floor Details for that floor.

Expected behaviour. Supports text filtering by floor name for fast access in larger homes.

User interactions. Tap to select, swipe to reveal quick edit/delete actions.

Future improvements. Add alphabetical/usage-based sorting options.

12.4 Floor Details

Purpose. Shows the rooms belonging to a selected floor in a responsive grid, per Section 6.

Components. Floor name header, room grid (`LazyVerticalGrid`), per-room device-count badges.

Navigation. Tapping a room card navigates to the Floor Plan screen for that room; back navigation returns to the Dashboard.

Expected behaviour. Room cards update live to reflect the number of devices currently ON.

User interactions. Tap to open a room, overflow menu to add/rename/delete a room.

Future improvements. Add a floor-level “all rooms off” action and room reordering.

12.5 Floor Plan

Purpose. Provides an interactive 2D spatial view of a room’s devices, positioned according to their normalized x/y coordinates, as described in Section 6.

Components. Simplified room outline, device icons positioned on the plan, tap targets for each device.

Navigation. Tapping a device icon navigates to Device Details.

Expected behaviour. Device icons visually reflect current state (e.g. a light icon glows amber when ON).

User interactions. Tap a device to view details; drag-and-drop (in edit mode) to reposition a device on the plan.

Future improvements. Support importing a real floor-plan image as the background layer.

12.6 Device Details

Purpose. Displays full status and controls for a single device, with a layout that adapts to the device's type (Section 7).

Components. Device name/type header, status chip, type-specific control widgets (toggle, per-switch toggles, schedule picker, snapshot button), a mini history/usage summary.

Navigation. Back navigation returns to the Floor Plan or Floor Details screen depending on entry point.

Expected behaviour. Control actions write directly to the Realtime Database and the UI reflects the resulting state change as soon as the listener fires, typically within a fraction of a second.

User interactions. Toggle ON/OFF, adjust schedule times (Smart Light), toggle individual switches (Multi-Switch Panel), request snapshot (Camera), rename or remove the device.

Future improvements. Add per-device notification preferences and a live sparkline of recent state changes.

12.7 Add Device

Purpose. Allows the user to register a new (simulated) device within a room.

Components. Device type selector, name field, room/floor confirmation, type-specific configuration fields (e.g. switch count for a Multi-Switch Panel, maximum duration for an Electric Iron).

Navigation. Presented as a modal/full-screen dialog over Floor Plan or Floor Details; dismiss or save returns to the calling screen.

Expected behaviour. On save, a new device node is created under `/devices` and referenced from the selected room.

User interactions. Select device type, fill configuration fields, tap Save or Cancel.

Future improvements. Add device templates and bulk-add for Multi-Switch Panels.

12.8 Reports

Purpose. Presents daily, weekly, and monthly usage summaries as described in Section 8.

Components. Period selector (Daily/Weekly/Monthly), bar chart of running time, pie chart of power usage proportion, safety-events list.

Navigation. Accessible from the Dashboard app bar; back navigation returns to the Dashboard.

Expected behaviour. Charts re-render when the selected period changes or when new history data arrives.

User interactions. Switch report period, tap a chart segment to drill into a specific device's history.

Future improvements. Add CSV/PDF export and year-over-year comparison views.

12.9 Notifications

Purpose. Displays the in-app history of alerts described in Section 9.

Components. Chronological list of alert cards (icon, message, timestamp, affected device link).

Navigation. Tapping an alert navigates to the relevant Device Details screen.

Expected behaviour. New alerts appear at the top of the list in real time as they are written to `/alerts`.

User interactions. Tap to view the related device, swipe to dismiss/mark as read.

Future improvements. Add per-category notification filtering and mute/snooze controls.

12.10 Settings

Purpose. Exposes application-level preferences.

Components. Notification toggle, theme (light/dark) selector, default iron maximum-duration preference, account/sign-out section (if Firebase Authentication is enabled), app version/about section.

Navigation. Accessible from the Dashboard app bar; back navigation returns to the Dashboard.

Expected behaviour. Preference changes are applied immediately and, where relevant, written to the user's profile node.

User interactions. Toggle switches, tap to open sub-dialogs (e.g. default duration picker).

Future improvements. Add per-household member permission management once multi-user support (Section 21) is introduced.

13 Navigation Flow

13.1 Primary Navigation Path

Figure 12 shows the primary linear navigation path through the application, from launch to the deepest common screens.

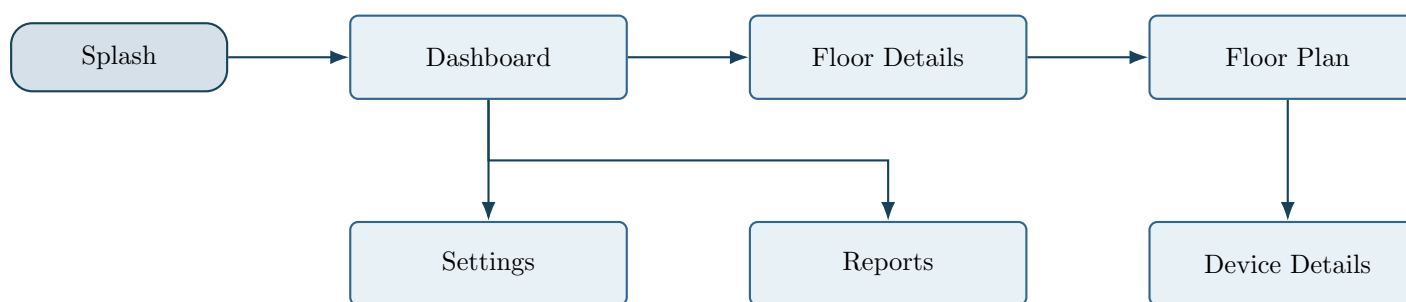


Figure 12: Primary navigation flow of the Luma Android application.

13.2 Navigation Architecture

Navigation is implemented using **Jetpack Navigation Compose**. A single `NavHost` is hosted in `MainActivity`, with each screen registered as a composable destination in the `navigation/` package (Section 5). Screens that require a parameter (Floor Details, Floor Plan, Device Details) receive it as a typed navigation argument (e.g. `floorId`, `roomId`, `deviceId`) encoded into the route string, which allows the corresponding `ViewModel` to be constructed with the correct Firebase path via a `SavedStateHandle`. The Dashboard, Reports, Settings, and Notifications screens are also reachable as top-level destinations from a persistent navigation drawer / bottom navigation bar, allowing the user to jump between these sections without returning to the Dashboard first.

13.3 Use Case Diagram

Figure 13 presents the primary use cases available to the homeowner (the sole actor for this mini project, given that multi-user support is a future improvement).

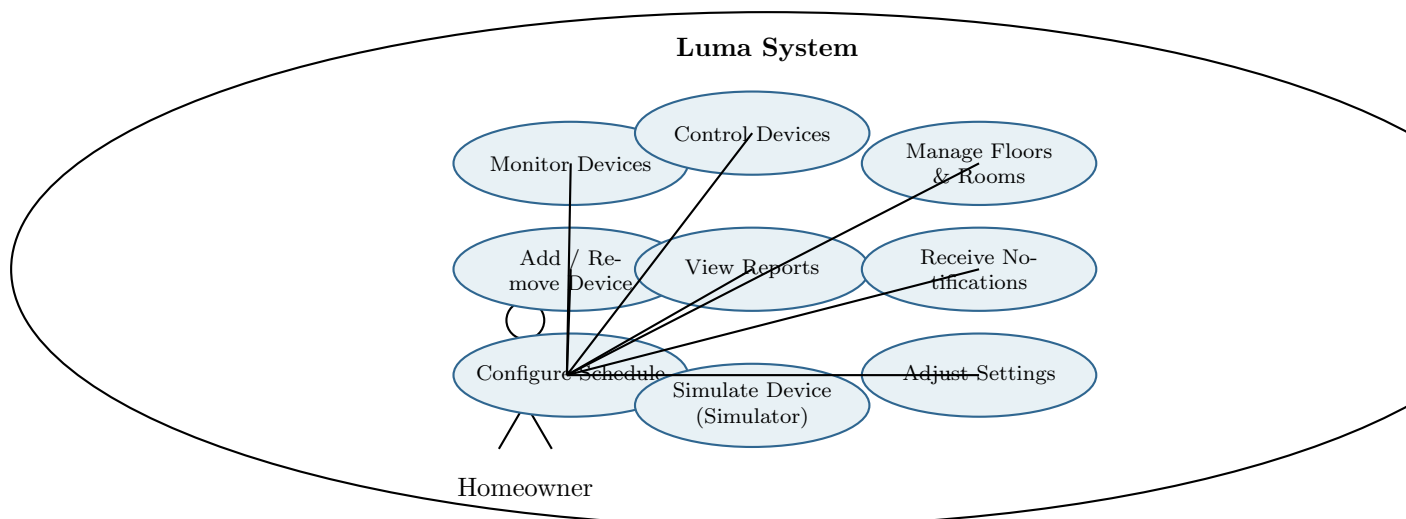


Figure 13: Use case diagram for the Luma system, with the Homeowner as the primary actor. “Simulate Device” is exercised through the separate Hardware Simulator application during development and demonstration.

13.4 Sequence Diagram: Controlling a Device

Figure 14 shows the sequence of interactions when the user toggles a device from the Android application and the change propagates to the Hardware Simulator.

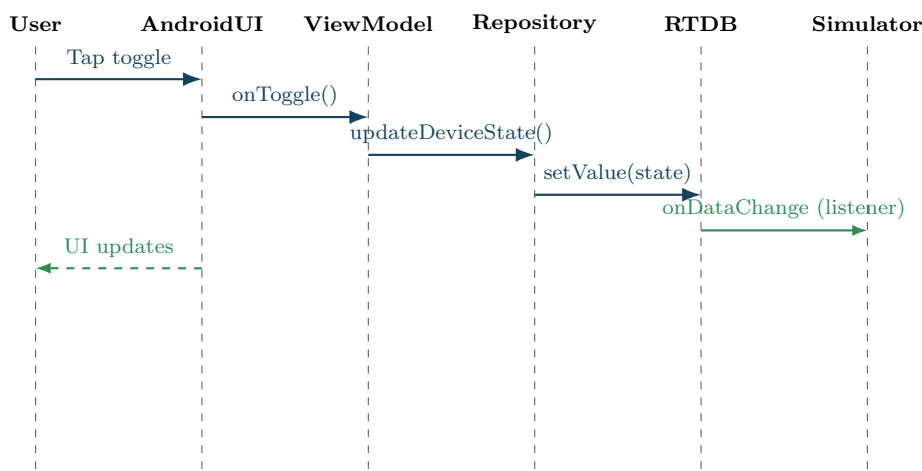


Figure 14: Sequence diagram for a device control action, from user tap through to the Hardware Simulator observing the resulting state change.

13.5 Activity Diagram: Electric Iron Safety Automation

Figure 15 models the activity flow of the Electric Iron’s automatic-shutdown safety feature, described functionally in Section 7 and implemented in Section 15.

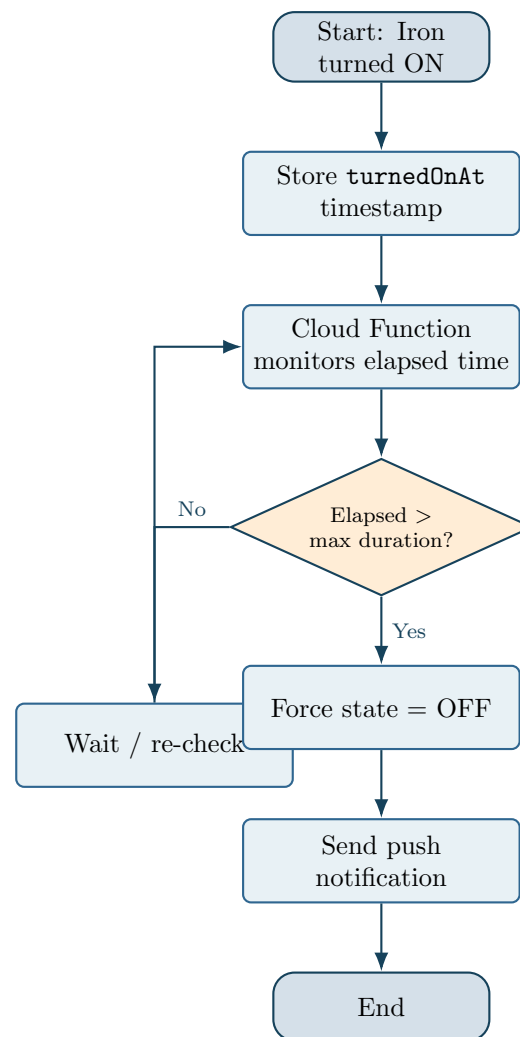


Figure 15: Activity diagram for the Electric Iron automatic-shutdown safety feature.

13.6 UML Class Diagram

Figure 16 shows a simplified class diagram of the core Android domain and data-access classes.

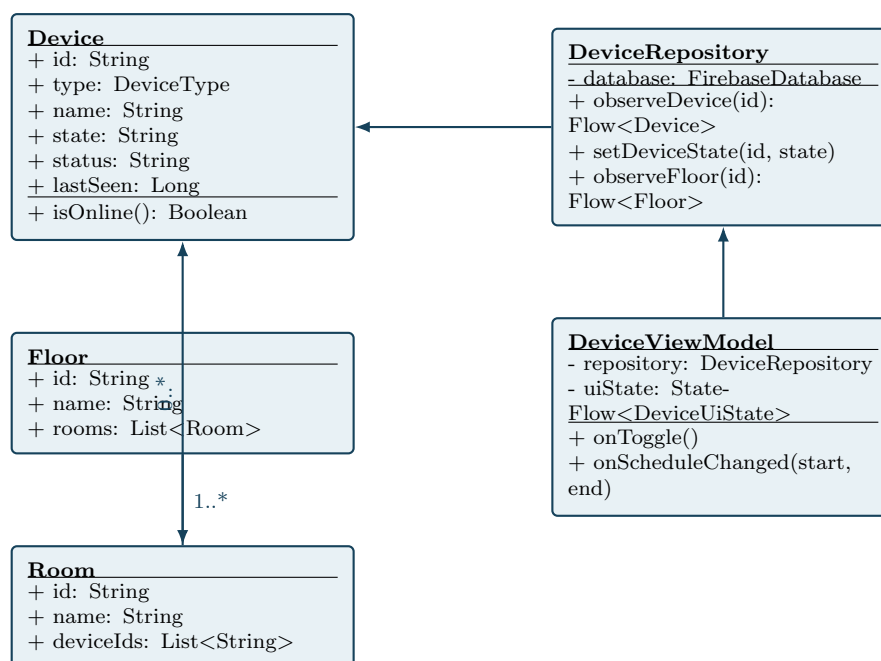


Figure 16: Simplified UML class diagram of the core Android domain and repository classes.

14 Firebase Synchronization

14.1 Bidirectional Synchronization Model

Because the Android application and the React Hardware Simulator never communicate directly, all synchronization is mediated by the Firebase Realtime Database’s listener mechanism. Figure 17 and Figure 18 show the two symmetric synchronization paths.

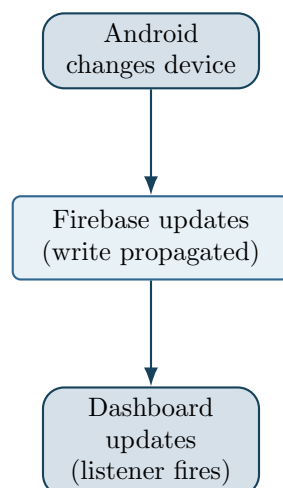


Figure 17: Synchronization path: Android application to Hardware Simulator Dashboard.

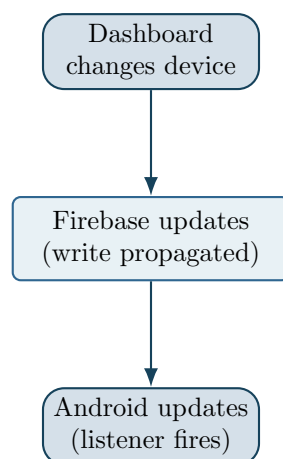


Figure 18: Synchronization path: Hardware Simulator Dashboard to Android application.

14.2 Realtime Database Listeners

On Android, synchronization is implemented using the Firebase Realtime Database’s `ValueEventListener` and `ChildEventListener` APIs, wrapped by the `DeviceRepository` class (Figure 16) into a Kotlin Flow using `callbackFlow`, so that ViewModels can consume updates using standard coroutine collection (`collectAsState` in Compose). On the React dashboard, the equivalent behaviour is achieved using the Firebase JavaScript SDK’s `onValue` listener, wrapped inside custom hooks such as `useDeviceState`, which internally call `onValue` on mount and detach the listener with the returned unsubscribe function on unmount.

A listener is attached at the narrowest path that satisfies a given screen’s needs – for example, the Device Details screen listens only to `/devices/{deviceId}` rather than the entire `/devices` node – to minimize bandwidth and re-render frequency.

14.3 Consistency and Conflict Handling

Because the Realtime Database applies writes in the order they are received per client connection and broadcasts the resulting value to all listeners, the system exhibits *eventual consistency with last-write-wins* semantics at the level of an individual field. For Luma’s use case – a single household with, in practice, one active operator at a time – this is an acceptable trade-off. Where atomicity across multiple fields is required (for example, when the iron auto-shutdown Cloud Function must simultaneously set `state` to OFF and clear `turnedOnAt`), the implementation uses Firebase’s multi-path `update()` call, which is applied atomically across all specified paths.

15 Firebase Cloud Functions

15.1 Role of Cloud Functions

Firebase Cloud Functions provide the only server-side logic in Luma, executing in Google’s managed Node.js runtime in response to Realtime Database triggers (`onValueWritten` / `onValueUpdated`) rather than in response to direct HTTP calls from either client. This event-driven model is what allows Luma to avoid a custom REST API entirely, as discussed in Section 3.

15.2 Iron Automatic Shutdown Function

The most safety-critical Cloud Function in Luma monitors every Electric Iron device for excessive continuous operation. Figure 19 shows the end-to-end flow, from the user switching the iron on through to all clients being updated.

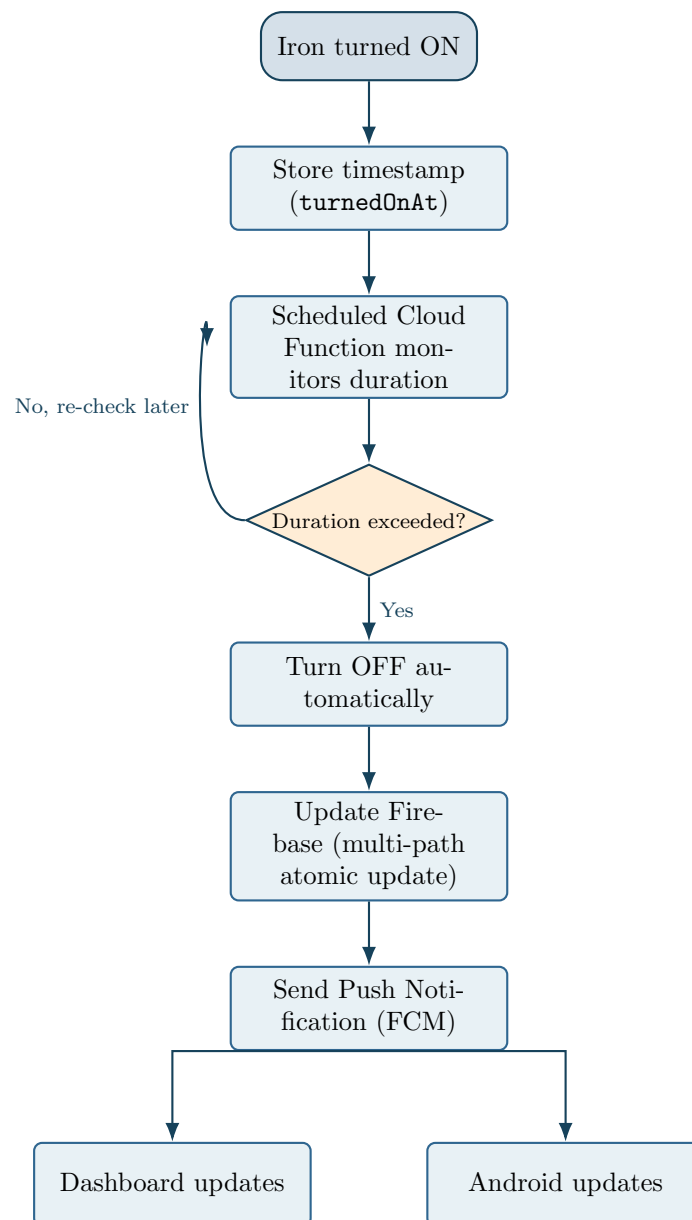


Figure 19: Flowchart of the Firebase Cloud Function implementing the Electric Iron automatic-shutdown safety feature.

```

1 exports.checkIronDuration = functions.pubsub
2   .schedule("every 1 minutes")
3   .onRun(async () => {
4     const devices = await db.ref("devices").once("value");
5     devices.forEach((snap) => {
6       const d = snap.val();
7       if (d.type === "iron" && d.state === "ON") {
8         const elapsedMin = (Date.now() - d.turnedOnAt) / 60000;
9         if (elapsedMin >= d.maxDurationMinutes) {
10          db.ref(`devices/${snap.key}`).update({
11            state: "OFF",
12            status: "OFF",
13            turnedOnAt: null
14          });
15          db.ref("alerts").push({
16            deviceId: snap.key,
17            type: "AUTO_SHUTDOWN",

```

```
18         message: "Iron automatically turned OFF after " +
19             d.maxDurationMinutes + " minutes",
20         timestamp: Date.now()
21     });
22     sendPushNotification(d.ownerFcmToken, snap.key);
23 }
24 }
25 });
26 });
```

Listing 4: Simplified pseudocode for the iron auto-shutdown Cloud Function.

15.3 Other Cloud Functions

- **Disconnection Detector.** A scheduled function that periodically compares each device's `lastSeen` timestamp against the current time and marks the device as `DISCONNECTED` if the threshold is exceeded, raising an alert as described in Section 9.
- **Smart Light Scheduler.** A scheduled function that evaluates each Smart Light's configured `startTime/endTime` against the current time and toggles the light's state accordingly.
- **Report Aggregator.** A function triggered whenever a device's `history` node receives a new `ON→OFF` entry, which recomputes the affected device's daily/weekly/monthly totals under `/reports`.
- **Alert Notifier.** A generic function triggered on any write to `/alerts` that forwards the alert to Firebase Cloud Messaging, used by both the disconnection detector and the iron auto-shutdown function.

16 MVVM Architecture

16.1 Overview

The Android application follows the **Model-View-ViewModel (MVVM)** pattern, which is the architecture recommended by Google for Jetpack Compose applications. Figure 20 shows the flow of data and events through the four architectural layers used in Luma.

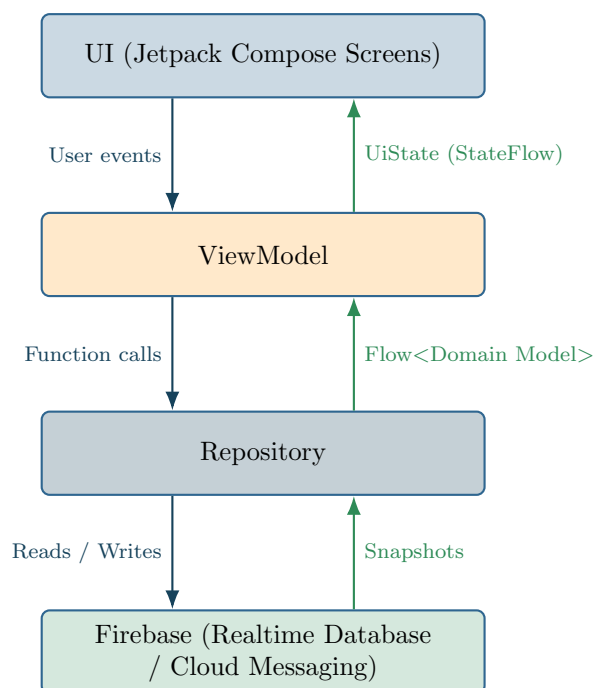


Figure 20: MVVM architecture as applied in the Luma Android application.

16.2 Layer Responsibilities

Layer	Responsibility
Model	Plain Kotlin data classes (<code>Device</code> , <code>Floor</code> , <code>Room</code> , <code>Alert</code> , <code>Report</code>) representing the domain entities described in Section 11, independent of any Firebase- or UI-specific types.
View (Compose UI)	Stateless composables that render a <code>UiState</code> object exposed by the corresponding <code>ViewModel</code> and forward user interactions (taps, toggles, text entry) as function calls back to the <code>ViewModel</code> ; the View never talks to Firebase directly.
ViewModel	Holds and exposes UI state as a Kotlin <code>StateFlow</code> , using <code>viewModelScope</code> coroutines to collect <code>Flows</code> from the <code>Repository</code> and translate domain models into UI-ready state; also validates and forwards user actions to the <code>Repository</code> .
Repository	Wraps the Firebase SDK, exposing a clean, testable Kotlin API (Section 5) that hides Firebase-specific listener and snapshot-parsing details from the rest of the app, and converts raw Firebase snapshots into domain model instances.

Table 9: Responsibilities of each MVVM layer in the Luma Android application.

16.3 Benefits Realized

- **Testability.** `ViewModels` can be unit-tested by supplying a fake `DeviceRepository` implementation, without requiring a real Firebase connection.
- **Separation of concerns.** UI code contains no Firebase-specific logic, and Firebase-specific code contains no UI logic, which keeps each layer focused and independently modifiable.

- **Configuration-change resilience.** Because `StateFlow` is held in the `ViewModel` (which survives configuration changes such as screen rotation), the UI does not need to re-fetch data from Firebase every time the screen is recomposed.
- **Consistency with Compose’s reactive model.** `StateFlow` integrates directly with Compose’s `collectAsStateWithLifecycle()`, giving automatic, lifecycle-aware recomposition whenever the underlying Firebase data changes.

17 Project Development Roadmap

17.1 Implementation Phases

The project was implemented across thirteen phases, summarized in Table 10.

Phase	Name	Description
1	Requirement Analysis	Define objectives, functional requirements, device types, and success criteria for the mini project.
2	UI Wireframes	Produce low-fidelity wireframes for all Android screens and the Hardware Simulator Dashboard.
3	Android Project Setup	Initialize the Android Studio project, configure Gradle, add Jetpack Compose and Firebase dependencies.
4	Navigation	Implement the Navigation Compose graph and route definitions for all ten screens.
5	Firebase Setup	Create the Firebase project, configure the Realtime Database, and define initial Security Rules.
6	Realtime Synchronization	Implement Repository classes and Realtime Database listeners on both Android and React.
7	Device Management	Implement Add/Edit/Delete for floors, rooms, and devices, and per-device-type control UI.
8	Reports	Implement usage-history aggregation and the Reports screen with bar/pie charts.
9	React Hardware Simulator	Build the standalone React dashboard that emulates physical devices.
10	Cloud Functions	Implement the iron auto-shutdown, disconnection detector, scheduler, and report aggregator functions.
11	Notifications	Implement Firebase Cloud Messaging integration and the in-app Notifications screen.
12	Testing	Execute the unit, UI, integration, and synchronization test plan described in Section 19.
13	Documentation	Produce this report and accompanying code documentation.

Table 10: Thirteen-phase implementation roadmap for Luma.

17.2 Project Timeline

Figure 21 presents the roadmap as a Gantt-style timeline across a 12-week mini-project schedule.

"Requirement Analysis", "UI Wireframes", "Android Setup", "Navigation", "Firebase Setup", "Realtime Sync", "Device Mgmt", "Reports", "Re

Figure 21: Gantt-style project timeline across a 12-week mini-project schedule.

17.3 Recommended Development Order

Table 11 presents the recommended step-by-step development order used by the team, together with the milestone and expected output of each step.

#	Step	Milestone	Expected Output
1	Android project creation	Project builds and runs an empty Activity	A configured Android Studio project with Compose and Gradle set up
2	UI development	Static screens render with placeholder data	Compose screens for all ten identified destinations
3	Navigation	User can move between all static screens	A working <code>NavHost</code> with all routes registered
4	Data models	Domain classes compile and are serializable	<code>Device</code> , <code>Floor</code> , <code>Room</code> , <code>Alert</code> , <code>Report</code> data classes
5	Dashboard	Floor list renders from hard-coded data	Dashboard screen with add/edit/delete floor dialogs
6	Floor management	Rooms and floor plan render correctly	Floor Details and Floor Plan screens
7	Device management	All five device types have working controls	Device Details and Add Device screens
8	Firebase integration	App reads/writes to a live Firebase project	Firebase project configured with initial Security Rules
9	Realtime synchronization	Two clients see the same state change live	Repository classes with Realtime Database listeners
10	React simulator	Simulator toggles devices and Android reflects it	Standalone React app with Device Cards and Status Chips
11	Cloud Functions	Iron auto-shuts-off correctly in testing	Deployed Cloud Functions for automation
12	Reports	Charts display aggregated usage data	Reports screen with bar/pie charts

#	Step	Milestone	Expected Output
13	Notifications	Push notifications arrive on safety events	FCM integration and Notifications screen
14	Testing	All test cases in Section 19 pass	Completed test execution log
15	Documentation	Report and code comments finalized	This report and inline code documentation

Table 11: Recommended development order with milestones and expected outputs.

18 Team Responsibilities

Luma was developed by a team of three members, with responsibilities divided along the natural boundaries between the Android application, the Firebase backend, and the React Hardware Simulator.

Member	Primary Responsibilities
Member 1	Android UI (Jetpack Compose screens for all ten destinations), Navigation Compose graph, Dashboard implementation, Reports screen and charts.
Member 2	Firebase project configuration, Realtime Database schema design and Security Rules, Realtime synchronization (Repository and listener implementation), Device Management screens and logic.
Member 3	React Hardware Simulator Dashboard, Firebase Cloud Functions (automation logic), Testing (unit, integration, and synchronization test execution), Documentation (this report).

Table 12: High-level responsibility allocation across the three team members.

18.1 Responsibility Matrix

Table 13 cross-references each major project deliverable against the team member primarily responsible for it (P) and any member providing secondary support (S).

Deliverable	Member 1	Member 2	Member 3
Android UI screens	P	S	–
Navigation graph	P	–	–
Dashboard & Floor Management	P	S	–
Reports screen & charts	P	–	S
Firebase schema & Security Rules	S	P	–
Realtime synchronization (Repository/listeners)	S	P	S
Device Management (Add/Edit/Delete)	S	P	–
React Hardware Simulator	–	S	P
Cloud Functions (automation)	–	S	P

Deliverable	Member 1	Member 2	Member 3
Notifications (FCM integration)	S	S	P
Testing (all categories)	S	S	P
Documentation (this report)	S	S	P

Table 13: Responsibility matrix: P = Primary owner, S = Secondary support.

19 Testing

A multi-layered testing strategy was used to validate Luma across the Android application, the Firebase backend, and the React Hardware Simulator.

19.1 Testing Categories

Category	Description
Unit Testing	Kotlin unit tests for ViewModels (using fake Repository implementations) and JavaScript unit tests for React hooks and utility functions.
UI Testing	Jetpack Compose UI tests (<code>createComposeRule</code>) verifying that toggles, dialogs, and navigation actions behave as expected on Android; React Testing Library assertions for the simulator's Device Cards.
Integration Testing	Tests that exercise a Repository against a real (or emulated) Firebase project to confirm that reads/writes and listener callbacks behave correctly end-to-end.
Realtime Synchronization Testing	Manual and scripted tests that change a device's state on one client and assert that the change is observed on the other client within an acceptable latency window.
Firebase Testing	Validation of Security Rules using the Firebase Emulator Suite's Rules Unit Testing library, confirming that unauthorized reads/writes are correctly rejected.
Cloud Function Testing	Unit tests for Cloud Function handlers using the Firebase Functions Test SDK, and manual verification of the iron auto-shutdown timing behaviour.
Dashboard Testing	Manual exploratory testing of the React Hardware Simulator across supported browsers, confirming device cards render and toggle correctly.
User Acceptance Testing	Structured walkthroughs of each functional requirement in Section 6 performed by team members acting as end users.

Table 14: Testing categories applied to the Luma project.

19.2 Representative Test Cases

ID	Test Case	Expected Result	Category
TC-01	Toggle a Smart Light ON from the Android app	Light state changes to ON in Firebase and on the Simulator within 1s	Synchronization
TC-02	Turn Electric Iron ON and wait beyond max duration	Iron is automatically turned OFF and a push notification is received	Cloud Function
TC-03	Simulator marks a device as disconnected	Android Notifications screen shows a “Device disconnected” alert	Cloud Function
TC-04	Add a new floor from the Dashboard	New floor node appears under /floors and is visible on both clients	Integration
TC-05	Attempt to write to /devices without authentication (where enabled)	Write is rejected by Firebase Security Rules	Firebase Rules
TC-06	Toggle two switches on a 3-switch Multi-Switch Panel independently	Only the toggled switches change state; the third remains unaffected	UI
TC-07	View Reports screen after several ON/OFF cycles	Bar and pie charts reflect the aggregated running time correctly	UI / Reports
TC-08	Request a camera snapshot	A mock image URL is returned and displayed within the Device Details screen	Simulator

Table 15: Representative test cases and expected results.

20 Security

20.1 Firebase Security Rules

Access to the Realtime Database is governed entirely by declarative **Firebase Security Rules**, evaluated by Firebase’s servers on every read and write, independent of any client-side validation. This is essential in a Firebase-only architecture, since there is no custom API layer to otherwise enforce access control.

```

1 {
2   "rules": {
3     "floors": {
4       ".read": "auth != null",
5       ".write": "auth != null"
6     },
7     "devices": {
8       "$deviceId": {
9         ".read": "auth != null",

```

```

10     ".write": "auth != null",
11     ".validate": "newData.hasChildren(['type', 'state', 'status'])"
12   }
13 },
14 "alerts": {
15   ".read": "auth != null",
16   ".write": "auth != null || root.child('serviceAccount').exists()"
17 },
18 "reports": {
19   ".read": "auth != null",
20   ".write": false
21 }
22 }
23 }

```

Listing 5: Simplified Firebase Realtime Database Security Rules for Luma.

The **reports** node is deliberately marked read-only for all clients, since report entries are exclusively generated by the Report Aggregator Cloud Function (Section 15), which operates with elevated Admin SDK privileges that bypass Security Rules.

20.2 Authentication

Firebase Authentication is included as an optional layer. When enabled, a single shared household account (email/password or Google sign-in) is used, consistent with Luma’s single-user scope in this mini project; Section 21 discusses extending this to per-member accounts.

20.3 Realtime Security Considerations

- Security Rules are validated using the Firebase Emulator Suite before deployment, as described in Section 19, to catch overly permissive or overly restrictive rules before they reach production.
- **.validate** rules enforce that device writes contain the expected required fields, reducing the risk of malformed data reaching either client.
- Because the Hardware Simulator and the Android app share the same authenticated identity in this mini project, Security Rules do not need to distinguish between the two client types; a production system would extend this with per-device-type or per-role rules.

20.4 Cloud Function Security

Cloud Functions execute using the Firebase Admin SDK, which operates with full read/write privileges to the Realtime Database, bypassing Security Rules entirely. Because Cloud Functions are only triggered by database events or an internal scheduler (never by direct client HTTP requests in this project), there is no publicly callable endpoint for an attacker to target, which reduces the attack surface compared to a custom REST API.

20.5 Database Rules Summary

Node	Read Access	Write Access
/floors	Authenticated users	Authenticated users
/devices	Authenticated users	Authenticated users (validated schema)

Node	Read Access	Write Access
/alerts	Authenticated users	Authenticated users or the Admin SDK (Cloud Functions)
/reports	Authenticated users	Admin SDK only (Cloud Functions)

Table 16: Summary of database access rules by node.

21 Future Improvements

While Luma satisfies the objectives defined for this mini project, several enhancements were identified as valuable directions for future work beyond the current scope:

Improvement	Description
Voice Assistant Integration	Integrate with Google Assistant and Amazon Alexa to allow voice-based device control, using their respective smart home SDKs on top of the existing Firebase data model.
Real Hardware (ESP32 / MQTT)	Replace the React Hardware Simulator with genuine ESP32-based microcontrollers communicating over MQTT, bridged into the same Firebase Realtime Database structure, allowing the Android app to work unchanged.
Face Recognition	Extend the Camera device type with on-device or cloud-based face recognition for visitor identification and enhanced security alerts.
Energy Analytics	Build richer analytics on top of the existing /reports node, including historical trend analysis and cost estimation based on local electricity tariffs.
AI Energy Optimization	Apply machine learning to historical usage data to suggest or automatically apply energy-saving schedules.
Multiple Users	Extend Firebase Authentication to support multiple household members with individual accounts and role-based permissions (e.g. adult vs. guest access).
Smart Sensors	Add support for passive sensor device types (temperature, humidity, motion, door/window contact) alongside the existing controllable device types.
Remote Access	Formalize secure remote access patterns (already partially supported by Firebase’s cloud-hosted nature) with additional safeguards such as two-factor authentication for control actions performed away from the home network.
Weather Integration	Incorporate a weather API to inform automation rules, such as adjusting lighting schedules based on sunrise/-sunset times or local conditions.

Table 17: Identified future improvements beyond the current project scope.

22 Conclusion

Luma demonstrates that a fully functional, realtime, multi-client smart home platform can be built using a purely serverless, Firebase-centric architecture, without the need for a custom REST API, a dedicated application server, or physical IoT hardware. By separating the concerns of presentation (the Android application and the React Hardware Simulator), synchronization (the Firebase Realtime Database and its client SDKs), and automation (Firebase Cloud Functions and Cloud Messaging), the project achieves a clean, modular architecture that satisfies every objective defined in Section 2: live monitoring, responsive control, multi-floor and multi-device management, realtime synchronization across independent clients, safety automation for the Electric Iron, usage reporting, and a realistic substitute for physical hardware through the Hardware Simulator Dashboard.

The MVVM architecture adopted on the Android side, combined with the Repository pattern’s encapsulation of Firebase-specific logic, produced a codebase that is testable, maintainable, and consistent with current Android development best practices. On the backend, Firebase Cloud Functions proved sufficient to implement all required automation – most notably the safety-critical iron auto-shutdown feature – entirely through database-triggered and scheduled execution, validating the architectural decision documented in Section 3 to avoid a custom backend server.

Beyond meeting the immediate requirements of the SCS 3311 mini project, Luma’s architecture is directly extensible toward a production-grade smart home system: the React Hardware Simulator could be replaced by genuine ESP32/MQTT-based hardware without any change to the Android application or the Firebase data model, and the features outlined in Section 21 – multi-user support, voice assistants, and AI-driven energy optimization – can each be layered onto the existing foundation incrementally. The project therefore serves both as a complete academic deliverable and as a credible foundation for continued development.